| Name | ID |
|---|---|
| Osama Abutaha | S23108673 |

## Lab Exercises:

Complete the following exercises in order. For each exercise, show your code, the output, and a brief explanation of what the result means.

### Exercise 1 — Truth Table Analysis (Warm-up)

Without running any code, fill in the truth table for: Implication(And(P, Q), Or(P, Not(Q)))

| P | Q | P ∧ Q | P ∨ ¬Q | (P ∧ Q) → (P ∨ ¬Q) |
|---|---|---|---|---|
| T | T | T | T | T |
| T | F | F | T | T |
| F | T | F | F | T |
| F | F | F | T | T |

Then verify your answers using model check().

```
from logic import *

P = Symbol("P")
Q = Symbol("Q")

knowledge = Implication(
            And(P, Q), Or(P, Not(Q)))

print(model_check(knowledge, knowledge))
```

The Output:

```
(venv3081)
True
```

### Exercise 2 — Extend harry.py

Add a fourth character: Snape. Encode the following additional rules, then check what conclusions can be drawn:

- Harry met exactly one of: Hagrid, Dumbledore, or Snape (update the Or)
- If it is raining, Snape did not come outside
- Snape was seen outdoors today (add this as a new fact)

```
1  from logic import *
2
3  rain = Symbol("rain")
4  hagrid = Symbol("hagrid")
5  dumbledore = Symbol("dumbledore")
6  snape = Symbol("snape")
7
8  knowledge = And(
9      Implication(Not(rain), hagrid),
10     Or(hagrid, dumbledore, snape),
11     Not(And(hagrid, dumbledore)),
12     Not(And(hagrid, snape)),
13     Not(And(snape, dumbledore)),
14     Implication(rain, Not(snape)),
15     snape
16 )
17
18 print(knowledge.formula())
19 print(model_check(knowledge, rain))
```

The Output:

```
(venv3081)
True
```

The output is **True** because the knowledge base contains a logical contradiction (unsatisfiability). Line 9 (Implication(Not(rain), hagrid)) forces rain to be true to explain why Hagrid is absent, while the Snape rule simultaneously implies it cannot be raining. In formal logic, an impossible knowledge base entails that any query is true. If Line 9 is commented out, this contradiction is removed, and the model checker correctly returns **False**. This is the logically sound answer because if Snape is outdoors, it cannot be raining according to the rule Implication(rain, Not(snape)).

## Exercise 3 — New Clue Scenario

Modify clue.py with a completely new set of characters:

- Suspects: AliceSuspect, BobSuspect, CarolSuspect
- Rooms: office, garage, basement
- Weapons: poison, rope, candlestick

Encode the following clues:

1. Cards in your hand: Not(alice), Not(office), Not(poison)

2. Opponent showed you one of: Not(bob), Not(garage), Not(rope)

3. You separately learned: Not(candlestick), Not(basement)

```python
import termcolor

from logic import *

alice = Symbol("AliceSuspect")
bob = Symbol("BobSuspect")
carol = Symbol("CarolSuspect ")
Suspects = [alice, bob, carol]

office = Symbol("office")
garage = Symbol("garage")
basement = Symbol("basement")
Rooms = [office, garage, basement]

poison = Symbol("poison")
rope = Symbol("rope")
candlestick = Symbol("candlestick")
Weapons = [poison, rope, candlestick]

symbols = Suspects + Rooms + Weapons

def check_knowledge(knowledge):
    for symbol in symbols:
        if model_check(knowledge, symbol):
            termcolor.cprint(f"{symbol}: YES", "green")
        elif not model_check(knowledge, Not(symbol)):
            print(f"{symbol}: MAYBE")

knowledge = And(
    Or(alice, bob, carol),
    Or(office, garage, basement),
    Or(poison, rope, candlestick)
)

knowledge.add(And(
        Not(alice), Not(office), Not(poison)))

knowledge.add(Or(
    Not(bob), Not(garage), Not(rope)))

knowledge.add(Not(candlestick))
knowledge.add(Not(basement))


check_knowledge(knowledge)
```

The Output:

```
(venv3081) PS C:\G
CarolSuspect : YES
garage: YES
rope: YES
```

## Exercise 4 — Add a Third Clue to mastermind.py

Suppose a third guess [blue in pos 0, green in pos 1, red in pos 2, yellow in pos 3] receives the feedback: 0 correct positions. Add this as a new knowledge.add() statement.

```python
from logic import *

colors = ["red", "blue", "green", "yellow"]
symbols = []
for i in range(4):
    for color in colors:
        symbols.append(Symbol(f"{color}{i}"))

knowledge = And()

# Each color has a position.
for color in colors:
    knowledge.add(Or(
        Symbol(f"{color}0"),
        Symbol(f"{color}1"),
        Symbol(f"{color}2"),
        Symbol(f"{color}3")
    ))

# Only one position per color.
for color in colors:
    for i in range(4):
        for j in range(4):
            if i != j:
                knowledge.add(Implication(
                    Symbol(f"{color}{i}"), Not(Symbol(f"{color}{j}"))
                ))

# Only one color per position.
for i in range(4):
    for c1 in colors:
        for c2 in colors:
            if c1 != c2:
                knowledge.add(Implication(
                    Symbol(f"{c1}{i}"), Not(Symbol(f"{c2}{i}"))
                ))

knowledge.add(Or(
    And(Symbol("red0"), Symbol("blue1"), Not(Symbol("green2")), Not(Symbol("yellow3"))),
    And(Symbol("red0"), Symbol("green2"), Not(Symbol("blue1")), Not(Symbol("yellow3"))),
    And(Symbol("red0"), Symbol("yellow3"), Not(Symbol("blue1")), Not(Symbol("green2"))),
    And(Symbol("blue1"), Symbol("green2"), Not(Symbol("red0")), Not(Symbol("yellow3"))),
    And(Symbol("blue1"), Symbol("yellow3"), Not(Symbol("red0")), Not(Symbol("green2"))),
    And(Symbol("green2"), Symbol("yellow3"), Not(Symbol("red0")), Not(Symbol("blue1")))
))

knowledge.add(And(
    Not(Symbol("blue0")),
    Not(Symbol("red1")),
    Not(Symbol("green2")),
    Not(Symbol("yellow3"))
))

knowledge.add(And(
    Not(Symbol("blue0")),
    Not(Symbol("green1")),
    Not(Symbol("red2")),
    Not(Symbol("yellow3"))
))

for symbol in symbols:
    if model_check(knowledge, symbol):
        print(symbol)
```

The Output:

```
(venv3081
red0
blue1
yellow2
green3
```

### Does this allow model check to completely solve the puzzle? Report your findings and explain why or why not?

Yes, the puzzle is solved.

After adding the new constraint, the model_check function identifies exactly one color for each position.

## Exercise 5 — Scalability Analysis (Discussion)

Answer the following questions in your lab report:

1. How many symbols does puzzle.py use? How many truth-table rows does model check explore? (Formula: $2^n$ where n = number of symbols)

Number of Symbols: 4 people x 4 houses = 16 Symbols

Number of Rows: $2^n = 2^{16} = 65{,}536$

2. How many symbols does mastermind.py use? How many rows?

Number of Symbols: 4 colors x 4 positions = 16 Symbols

Number of Rows: $2^n = 2^{16} = 65{,}536$

3. If we extended mastermind.py to 6 colors and 6 positions, how many symbols would there be? How many rows?

Number of Symbols: 6 colors x 6 positions = 36 Symbols

Number of Rows: $2^n = 2^{36} = 68{,}719{,}476{,}736$

4. What algorithmic approach could replace truth-table enumeration for large problems? Name one and briefly explain how it improves on the exponential complexity

The DPLL algorithm replaces truth-table enumeration by using unit propagation and backtracking. Unlike the $2^n$ approach, it avoids exploring every possible combination by discarding impossible paths early, making it significantly faster for large-scale problems like the 6x6 Mastermind grid.